

Omilog: O-Expressions for the Omicron Object Model

A Specification for Addressed Cons Expressions, Source Blocks, and WLOG-Safe Rewrite Logic in `.omi` Files

Status: Draft Specification / Research-to-Canon Proposal

Project: Omi Object Model / Universal Life Protocol

Scope: `.omi` expression syntax, O-expressions, addressed declarations, `omi-` / `-imo` source blocks, cons dot notation, WLOG-style symmetry, closures, combinators, and lookup-function bodies.

Abstract

Omilog is the declarative expression language of the Omi Object Model.

It is not Prolog.

It is not Datalog.

It is not plain Lisp.

Omilog is an addressed, source-block-oriented cons expression system for `.omi` files.

Its basic unit is:

```
omi-address/prefix KEYWORD assignment

omi-
  o-expression body
-imo
```

The address line acts like a selector.

The `omi-` / `-imo` region acts like a declaration body.

The body may contain cons cells, association lists, emoji rewrite pairs, Unicode symbolic carriers, closure declarations, combinator declarations, or future non-printing traversal characters.

Omilog exists to let `.omi` files carry lookup functions, rewrite rules, closure bodies, and combinator definitions without confusing those expressions with runtime authority.

The core law remains:

```
Q_frame(S) validates.  
Omiolog expresses.  
Q_xy(x,y) projects.
```

An Omiolog expression may describe a rule, fact, closure, combinator, or cons structure. It may not validate an invalid frame by itself.

1. Why Omiolog Exists

The OMI system needs a notation that can express:

```
rules  
facts  
closures  
combinators  
cons cells  
association lists  
emoji rewrite rules  
source blocks  
lookup functions  
prefix-scoped declarations
```

Existing systems cover parts of this:

```
Lisp      → cons cells and symbolic expressions  
Prolog    → relations and unification  
Datalog   → finite declarative facts and rules  
CSS       → selector-scoped declaration blocks  
Org Babel → source blocks
```

But OMI needs all of these at once, under an address pointer.

Omiolog is therefore a language of **addressed expressions**.

Its purpose is not merely to compute. Its purpose is to bind expression bodies to OMI addresses.

2. The Basic Omiolog Record

An Omiolog record has two parts:

```
HEAD
BODY
```

The head is an OMI declaration line:

```
omi-address/prefix KEYWORD assignment
```

The body is an OMI source block:

```
omi-
  expression body
-imo
```

Example:

```
omi-0000-0000-0000-0000-0000-0000-0c05-0002/128 CONS omi-palindromic-mnemonic-
token

omi-
(
  (token . omi---imo)
  (dot . (omi . (--- . imo)))
  (expanded . (Omicron . (Tetragrammaton . Metatron)))
  (authority . projection-only)
  (forbids . token-authorizes-invalid-frame)
)
-imo
```

The address line selects the declaration.

The body explains the expression.

Together:

```
(address . (keyword . body))
```

This is Omilog's root form.

3. O-Expressions

An Omilog body is made of **O-expressions**.

An O-expression is a cons-inspired expression scoped by an OMI address.

The canonical dot form is:

```
(key . value)
```

Nested:

```
(key . (subkey . value))
```

Alist form:

```
((key . value)  
 (key2 . value2)  
 (key3 . value3))
```

OMI-prefixed symbolic form:

```
(omi-key . omi-value)
```

Emoji carrier form:

```
(omi-🔑 . omi-📄)
```

Addressed carrier form:

```
(omi-ca55-0000-0000-0000-0000-0004-0003 . derive-shadow-canvas)
```

O-expressions differ from ordinary Lisp expressions because they are not free-floating. They are always scoped by an OMI declaration head.

4. Why Not Just Lisp?

Lisp expressions are evaluated by a Lisp evaluator.

Omiolog expressions are not automatically evaluated.

They are parsed, indexed, and attached to an OMI address.

A Lisp expression such as:

```
(car x)
```

means “evaluate the `car` function on `x`.”

An Omiolog expression such as:

```
(car . Omicron)
```

means “declare a structural association between `car` and `Omicron` inside this addressed OMI record.”

Evaluation is a later phase.

Therefore:

```
Lisp expressions compute.  
O-expressions declare addressed structure.
```

Omiolog borrows cons notation but changes its authority model.

5. Why Not Prolog?

Prolog expresses facts and rules through relations and unification.

Example:

```
parent(alice, bob).  
ancestor(X, Y) :- parent(X, Y).
```

Omiolog does not begin with unification.

Omilog begins with an OMI pointer.

Example:

```
omi-0000-0000-0000-0000-0000-0000-c005-0021/128 CONS emoji-rewrite-pair-alist

omi-
(
  (rewrite .
    '((omi-🔧 . omi-🔑)
      (omi-🔑 . omi-🚗)
      (omi-🚗 . omi-👑)))
  (authority . projection-only)
)
-imo
```

This is not a Prolog relation.

It is an addressed lookup body.

Therefore:

```
Prolog asks: what relations are true?
Omilog asks: what expression body belongs to this OMI address?
```

6. Why Not Datalog?

Datalog is useful for finite relational facts and rules.

Omilog is broader.

It must support:

```
dot notation
source blocks
emoji
Unicode
control-character traversal
binary frame references
closure bodies
combinator bodies
```

```
CSSOM-like cascades  
cons/alist rewrite tables
```

Datalog is excellent for monotone facts.

Omilog is a structured declaration notation for the OMI file system.

Therefore:

```
Datalog grounds finite relation logic.  
Omilog carries addressed expression bodies.
```

Omilog may compile into Datalog-like facts later, but it is not identical to Datalog.

7. The Five Omilog Files

Omilog is distributed across five `.omi` files:

```
RULES.omi  
FACTS.omi  
CLOSURES.omi  
COMBINATORS.omi  
CONS.omi
```

Their roles are:

```
RULES declare.  
FACTS ground.  
CLOSURES seal.  
COMBINATORS compose.  
CONS reduce.
```

7.1 RULES.omi

Declares normative obligations.

```
omi-address/prefix MUST invariant-name
```

Rules say what must be true.

They do not claim implementation.

7.2 FACTS.omi

Grounds implementation.

```
omi-address/prefix FACT fact-name
```

Facts say what exists, is implemented, tested, documented, or projected.

They do not create new obligations.

7.3 CLOSURES.omi

Seals bounded states.

```
omi-address/prefix CLOSE closure-name
```

Closures say when a structure is complete.

They prevent open-ended interpretation.

7.4 COMBINATORS.omi

Declares lawful composition.

```
omi-address/prefix COMBINE combinator-name
```

Combinators describe how lawful parts compose.

They do not authorize state.

7.5 CONS.omi

Declares structural reduction.

```
omi-address/prefix CONS cons-name
```

CONS records describe pairing, nesting, dot notation, alists, and palindromic mnemonic forms.

They do not validate frames.

8. The OMI Source Block

The source block is:

```
omi-  
  body  
-imo
```

It is the Omilog equivalent of a CSS declaration block.

CSS:

```
selector {  
  property: value;  
}
```

Omilog:

```
omi-address/prefix KEYWORD assignment  
  
omi-  
  (  
    (property . value)  
  )  
-imo
```

Mapping:

```
CSS selector      → OMI address line  
CSS declaration   → omi- / -imo source block  
CSS property/value → cons pair  
CSS cascade       → OMI prefix-specific lookup  
CSSOM             → OMIOM
```

The source block is not runtime authority.

It is a declaration body.

9. The Palindromic Mnemonic

The core mnemonic token is:

```
omi---imo
```

It expands as:

```
(omi . (--- . imo))
```

And further as:

```
(Omicron . (Tetragrammaton . Metatron))
```

This maps the binary16 abstraction:

```
sign bit      → Omicron Sign-Bit  
exponent      → Tetragrammaton Exponent  
significand   → Metatron Significand
```

In Omilog:

```
Omicron heads.  
Tetragrammaton scales.  
Metatron witnesses.
```

The hyphens are mnemonic delimiters.

They do not validate a frame.

10. O-Expressions and F-Expressions

In Lisp history, an F-expression receives its arguments unevaluated.

That idea matters for Omilog.

An O-expression should also be treated as unevaluated by default.

Example:

```
(rewrite .  
  '((omi-🚗 . omi-🔑)  
    (omi-👷 . omi-🚗)))
```

The quote means the rewrite table is carried as data.

It is not executed during parsing.

This gives Omilog its meta-circular property:

Omilog expressions describe future evaluation without forcing present evaluation.

Therefore:

F-expression = unevaluated function expression
O-expression = unevaluated addressed OMI expression

The parser preserves the form.

The compiler later decides whether and how to reduce it.

11. Omilog and WLOG

The name Omilog also resonates with “without loss of generality.”

This is not just a pun.

In OMI, many reductions depend on symmetry:

```
left/right  
source/shadow  
direct/folded  
omi/imo  
car/cdr  
subject/object  
valid/failure
```

A WLOG step is valid only when symmetry or equivalence is present.

Omiolog must preserve the same discipline.

An Omiolog rewrite may assume one orientation only if a closure proves the mirrored orientation is equivalent.

Example:

```
omi-0000-0000-0000-0000-0000-0000-c005-c010/128 CLOSE omi-imo-palindromic-
symmetry

omi-
(
  (requires . (token . omi---imo))
  (symmetry . (omi ↔ imo))
  (forbids . proof-by-single-unrepresentative-example)
  (permits . wlog-orientation-choice)
)
-imo
```

So Omiolog should define a **WLOG-safe rewrite rule**:

A rewrite may be taken without loss of generality only when a closure proves the symmetry that transports the proof to the omitted cases.

This is crucial.

Without closure, a rewrite is just an example.

With closure, it becomes WLOG-safe.

12. WLOG-Safe Rewrites

An Omiolog rewrite has three possible levels:

```
example
rewrite
wlog-safe rewrite
```

12.1 Example

A single case:

```
(omi-🛹 . omi-👉)
```

This only says skateboard maps to label in this local body.

12.2 Rewrite

A declared transformation:

```
(rewrite . ((omi-🛹 . omi-👉)))
```

This says the pair is part of a rewrite table.

12.3 WLOG-Safe Rewrite

A rewrite plus a closure:

```
(closure . four-fold-selector-symmetry)
```

Now the rewrite may generalize across the symmetry class.

Canonical rule:

```
No WLOG without closure.
```

Or:

```
WLOG requires symmetry closure.
```

13. Lookup Functions

Omilog source blocks are lookup-function bodies.

A lookup function is an addressed mapping:

source key → target value

Example:

```
omi-ca55-0000-0000-0000-0000-0000-0004-0003/128 MUST derive-shadow-canvas-from-
four-fold-selector

omi-
(
  (selector . omi-ca55-0000-0000-0000-0000-0000-0004-0003/128)
  (source . omi-0000-0000-0000-0000-0000-0000-0c05-0002/128)
  (rewrite .
    '((omi-🚲 . omi-👉)
      (omi-👉 . omi-🚲)
      (omi-🕒 . omi-👑)))
  )
-imo
```

This expresses a scoped lookup table.

It can later compile into:

```
hash map
trie
CSSOM-like rule table
Datalog facts
Prolog relations
JavaScript Map
WASM table
eBPF map
```

But the Omilog body itself remains declarative.

14. Emoji as O-Expression Symbols

Emoji may appear inside O-expressions as symbolic carriers.

Example:

```
(omi-🕒 . omi-👑)
```

The emoji token must resolve through vendor emoji data before being treated as a valid carrier.

The emoji validates as Unicode carrier data.

It does not validate an OMI frame.

Canonical distinction:

```
Unicode validates the emoji carrier.  
Q_frame validates the OMI frame.  
Omiolog attaches the expression to an address.  
Q_xy projects decoded state.
```

15. OMIOM: The Object Model

Omiolog produces an OMI Object Model, or OMIOM.

A parsed Omiolog record can become:

```
{  
  address,  
  prefixBits,  
  keyword,  
  assignment,  
  sourceBlock,  
  oExpressions,  
  authority,  
  lookupIndex  
}
```

This mirrors CSSOM.

CSSOM stores parsed CSS rules.

OMIOM stores parsed OMI declarations.

CSSOM applies selector specificity.

OMIOM applies prefix specificity.

CSS:

```
#id > .class > element
```

OMI:

```
/128 > /112 > /96 > /64 > /48 > default
```

Thus:

```
OMIOM = CSSOM for addressed cons/rewrite declarations.
```

16. Formal Grammar

A minimal Omilog grammar:

```
document      := record*
record        := head source_block?
head          := omi_address prefix keyword assignment
source_block  := "omi-" body "-imo"
body          := raw | o_expression*
o_expression  := atom | pair | list | quote
pair          := "(" atom "." body ")"
list          := "(" body* ")"
quote         := "'" body
keyword       := MUST | FACT | CLOSE | COMBINE | CONS | EQUALS
```

This grammar is intentionally permissive in v1.

The parser should preserve raw text.

Evaluation comes later.

17. Native Control Character Future

Visible block tags:


```
omi-  
-imo
```

are aliases.

Future native traversal may use control characters:

```
FS 0x1C → file/document boundary  
GS 0x1D → group boundary  
RS 0x1E → record/block opener  
US 0x1F → unit/block closer
```

So:

```
omi-  visible opener  
-imo  visible closer
```

may later normalize to:

```
RS ... US
```

This gives `.omi` files two modes:

```
human-visible mode  
native traversal mode
```

Both parse to the same OMIOM record shape.

18. Authority Boundary

Omilog must obey the OMI validation boundary.

```
Q_frame(S) validates the 128-bit carrier.  
Omilog structures the addressed declaration body.  
Q_xy(x,y) projects decoded state into geometry.
```

Forbidden:

```
emoji validates frame
omi---imo validates frame
hyphen count validates frame
dot notation validates frame
source block validates frame
color validates frame
```

Allowed:

```
emoji carries symbolic state
omi---imo names a cons mnemonic
hyphen count hints scale
dot notation declares structure
source block carries lookup body
color projects state
```

19. File-Level Roles

Omilog should be used differently in each file:

```
RULES.omi:
  Omilog body explains normative obligation.

FACTS.omi:
  Omilog body points to source/test/projection.

CLOSURES.omi:
  Omilog body proves bounded sealing.

COMBINATORS.omi:
  Omilog body declares pure function composition.

CONS.omi:
  Omilog body declares dot notation, alists, and reduction forms.
```

This prevents logical bleeding.

20. Implementation Plan

Phase 1 — Parser

Implement:

```
parseOmiClause()  
parseOmiDocument()  
parseOmiSourceBlock()
```

The parser attaches `omi- / -imo` bodies to the preceding record.

It preserves raw text.

It does not evaluate.

Phase 2 — OMIOM Index

Build an index:

```
address → records  
prefix → records  
keyword → records  
assignment → records
```

Phase 3 — O-Expression Reader

Add a permissive reader for:

```
pairs  
lists  
quotes  
atoms  
emoji atoms  
omi-address atoms
```

Phase 4 — Lookup Compiler

Compile selected O-expression bodies into lookup tables.

Phase 5 — Closure Checker

Allow WLOG-safe rewrites only when a closure exists.

Phase 6 — Projection

Route decoded symbolic state into:

```
Q_xy
local240
slot5040
DOM/CSSOM
```

21. Canonical Rules

```
# [Rule 0x84]: Omiolog O-Expression Source Language
omi-0000-0000-0000-0000-0000-0000-0084-0001/128 MUST parse-omiolog-o-expressions

# [Rule 0x85]: WLOG-Safe Rewrite Closure
omi-0000-0000-0000-0000-0000-0000-0085-0001/128 MUST require-closure-for-wlog-
safe-rewrite

# [Rule 0x86]: O-Expression Authority Boundary
omi-0000-0000-0000-0000-0000-0000-0086-0001/128 MUST preserve-o-expression-
authority-boundary

# [Rule 0x87]: OMIOM Prefix Cascade
omi-0000-0000-0000-0000-0000-0000-0087-0001/128 MUST resolve-omiolog-records-by-
prefix-specificity
```

22. Canonical Facts

Only after implementation:

```
omi-0000-0000-0000-0000-0000-0000-0084-1001/128 FACT omiolog-o-expression-parser-
implemented
omi-0000-0000-0000-0000-0000-0000-0085-1001/128 FACT wlog-safe-rewrite-closure-
tested
omi-0000-0000-0000-0000-0000-0000-0086-1001/128 FACT o-expression-authority-
```

```
boundary-tested
omi-0000-0000-0000-0000-0000-0000-0087-1001/128 FACT omiom-prefix-cascade-
implemented
```

23. Final Doctrine

Omilog is the O-expression language of OMI.

It gives `.omi` files a way to express:

```
addressed declarations
lookup functions
cons structures
alist tables
emoji rewrite pairs
closure bodies
combinator bodies
WLOG-safe rewrite symmetry
```

Its final doctrine is:

```
OMI address selects.
omi- opens.
O-expression declares.
-imo closes.
Closure permits WLOG.
Q_frame validates.
Q_xy projects.
```

24. One-Sentence Summary

Omilog is the addressed O-expression language of the Omi Object Model: a CSSOM-like, Lisp-inspired, WLOG-safe declaration system where OMI addresses select `omi- / -imo` source blocks, source blocks carry cons/alist/emoji rewrite bodies, closures justify symmetry-based generalization, and no expression becomes runtime authority unless it is grounded by pointer, rule, implementation, test, and replay/projection path.